EDUMATCHER · OPERATOR TRAINING

Drop Copy

The audit and clearing feed of the exchange

A ground-up guide for new trading system operators: what Drop Copy is, what it carries, how it is delivered, the tools that read it, and every configuration knob that changes its behaviour.

10
questions answered

3
ways to receive it

2
tools provided

Sources: 200-drop-copy · 201-dc-gateway · 202-dc-spy-cli · 010-configuration

BEFORE WE START

Who this deck is for, and what you need to know first

No prior exchange-infrastructure experience is assumed. Every term is defined the first time it appears.

A

You are a system operator

You start, stop, configure and troubleshoot the exchange processes. You do not need to write code to follow this deck.

B

Four terms, up front

Engine = matches orders. Gateway = a door into the system. Fill = a completed trade for one side. Feed = a continuous stream of messages.

C

How it is structured

Ten questions, in order. The final third is a technical reference you can return to when configuring or debugging.

i

Why Drop Copy deserves a whole deck

It is how auditors, risk systems and clearing/settlement see what the exchange did. If it is misconfigured, the trading still works — and nobody downstream notices anything is missing until reconciliation fails.

EduMatcher is an educational exchange. Where its behaviour differs from a production venue, the deck says so explicitly.

AGENDA

Ten questions, answered in order

01	What Drop Copy is for
02	What information it carries
03	When it is created and sent
04	The format on the wire
05	How ALF relates to Drop Copy
06	Using the DC Gateway
07	The DC Spy client
08	Good-to-know operating facts
09	Errors the feed can give
10	Configuration that changes it

Slides 40+ form a standing reference: protocol tables, error codes and every configuration field.

01

What is Drop Copy for?

Trading systems must tell more than one audience about a trade. Drop Copy is the second, independent telling — the one auditors and clearing rely on.

- The problem it solves
- The definition
- Who consumes it
- Where it sits in the architecture

THE PROBLEM

One delivery path is not enough

The trader's own connection tells the trader about their fill. Everyone else needs to hear about it too — reliably, and even when that connection is gone.

Risk must still see the fill

A risk system that only learns of fills through the trading gateway goes blind exactly when a gateway fails — the moment risk matters most.

Compliance wants one stream

A compliance monitor should see every fill from every participant in a single ordered stream — not one connection per gateway to stitch together.

Back office needs proof of gaps

Settlement cannot silently miss a trade. It needs numbered messages so a missing one is detectable, not merely unlikely.

DEFINITION

A copy of every execution, dropped to a side channel

Drop Copy is a secondary stream of trade and fill notifications, sent to risk, compliance and back-office systems in parallel with the primary trade confirmation path.

The word "drop"

As each execution report is processed, a copy is dropped onto a side channel. The recipient never has to be part of the order flow to receive it — it is not their trade, it is a copy of the record.

Independent of the trader
Delivered whether or not the trading session is up.

Read-only, always
Nothing on this channel can place, change or cancel an order.

* **The one-sentence version for a new operator**
Drop Copy is the exchange saying "this trade happened" a second time, on a channel built for people who watch rather than trade.

CONSUMERS

Who is on the other end of the feed

Four audiences, each with a different reason for needing an independent copy.

Risk management

Tracks live exposure per participant. Needs fills within milliseconds and cannot depend on the trading connection staying up.

Compliance & audit

Reconstructs what happened, in order, across every participant. Sequence numbers make the record provably complete.

Clearing & settlement

Turns fills into obligations to be settled. A missed fill is a real financial break, so gap detection is mandatory.

Operations tooling

Dashboards and spy tools that make live trading observable without touching the order path.

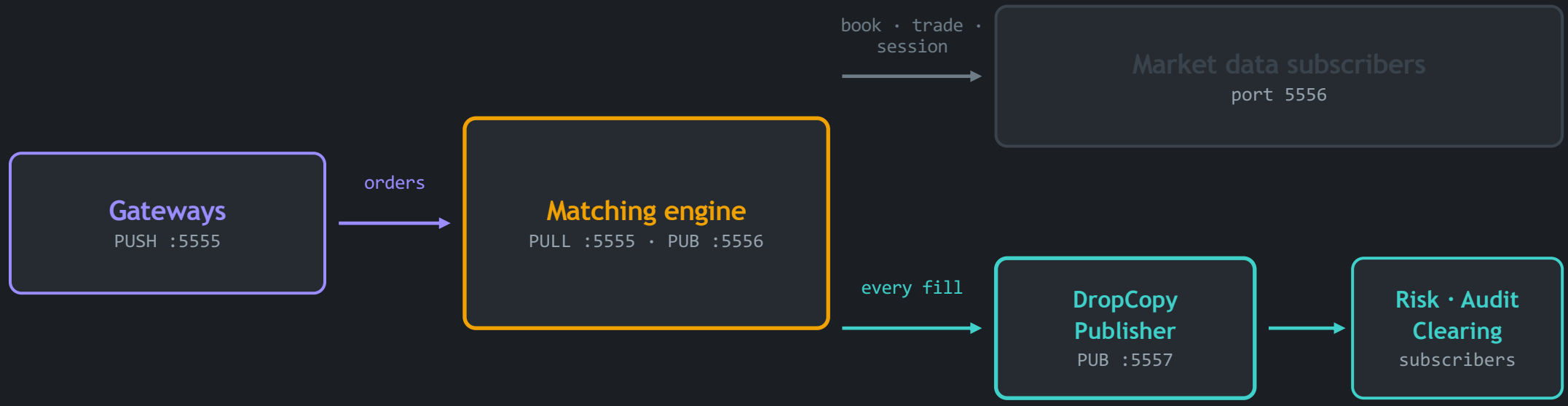
Common thread: all four are observers. None of them submits orders, and none should be able to.

* **Why they are grouped together**
Every one of them needs completeness over speed of interaction: they would rather receive a numbered, replayable record than be able to answer back.

ARCHITECTURE

Where Drop Copy sits inside EduMatcher

The engine already broadcasts market and trade events on one socket. Drop Copy is a separate, dedicated socket next to it.



Two separate broadcasts: general market events on 5556, per-fill drop copy on 5557. They never share a socket.

Socket	Address	Purpose
Engine PULL	`tcp://127.0.0.1:5555`	Engine receives orders from gateways
Engine PUB	`tcp://127.0.0.1:5556`	Book snapshots, trades, session state
Drop copy PUB	`tcp://127.0.0.1:5557`	Fill events to risk / audit / clearing

DESIGN CHOICES

Three decisions that explain the feed's behaviour

These are not implementation trivia — each one produces a symptom you will meet as an operator.

	WHAT IT DOES	WHAT IT MEANS FOR YOU
Bound late, in the run loop	The publisher binds its socket when the engine starts running, not when it is constructed.	So: a fresh engine that never entered its run loop holds no port — and nothing is published.
A failed bind does not stop trading	If port 5557 is already in use, the engine logs a warning and carries on without a Drop Copy publisher.	So: the exchange can be happily matching trades while the audit feed is silent. Always check the engine log at startup.
Closed explicitly on shutdown	The publisher is closed cleanly when the engine stops, releasing the socket.	So: a clean shutdown frees 5557 for the next run; a hard kill may not.

NAME COLLISION

Drop Copy feed vs. the RALF DROP_COPY channel

Two different things share a name. Confusing them is the single most common misunderstanding for new operators.

Engine Drop Copy feed

This deck

- Published by the DropCopyPublisher on its own socket, port 5557
- Carries fills only, with sequence numbers and an in-memory replay buffer
- No role model — a subscriber filters by topic prefix
- Use it for a per-participant, sequenced, replayable fill feed

RALF DROP_COPY channel

A different mechanism

- One of three role-based channels on the RALF text protocol: CLEARING, DROP_COPY, AUDIT
- Sourced from the trade broadcast on port 5556 — not from 5557
- All three channels carry identical trade content; only the entitlement role differs
- Use it when you need role-gated access enforced per connection

Rule of thumb

If someone says "drop copy" and means port 5557, they mean this feed. If they mention CLEARING or AUDIT alongside it, they mean RALF's channels.

02

What information does it carry?

Every drop copy message is a small, flat record of one side of one fill — plus an envelope that makes the stream auditable.

- The two frames
- Envelope fields vs. event fields
- What is deliberately absent
- Sequence numbers and why they matter

ANATOMY

One message, two frames

A drop copy message is a two-part message: an addressing line, then the data.

FRAME 0 — TOPIC

`drop_copy.event.<gateway_id>`

The address label. Subscribers filter on the start of this string, so it decides who receives the message.

FRAME 1 — PAYLOAD

`{ "seq": 42, "symbol": "MSFT", ... }`

A JSON object. Encoded with a fast serializer when available, otherwise the standard one — the bytes on the wire are identical either way.

A COMPLETE MESSAGE, AS PUBLISHED

```
drop_copy.event.TRADER01
{"seq":42,"timestamp":1700000000000000000,"gateway_id":"TRADER01","event_type":"order.fill",
 "order_id":"ord-001","symbol":"MSFT","fill_qty":100,"fill_price":420.0,"liquidity_flag":"MAKER"}
```

Read it as: message number 42, from gateway TRADER01, order ord-001 traded 100 MSFT at 420.00 as the resting (MAKER) side.

PAYLOAD FIELDS

The envelope and the event, in one flat object

Four envelope fields are always present. The event's own fields are merged in at the same level — there is no nesting.

ENVELOPE — ALWAYS PRESENT

Field	Type	Meaning
seq	int	Message number: starts at 1, only ever increases
timestamp	int	Nanosecond timestamp of the event
gateway_id	str	Which gateway submitted the order
event_type	str	Currently always order.fill

EVENT FIELDS — MERGED IN AT TOP LEVEL

Field	Example	Meaning
order_id	ord-001	The order that was filled
symbol	MSFT	Instrument traded
fill_qty	100	Quantity of this fill
fill_price	420.0	Display price as a decimal, not raw ticks
liquidity_flag	MAKER	Resting side, or TAKER for the aggressor

Three fields you might expect, and will not get

There is no remaining_qty, no side (buy/sell) and no leaves_qty in the published payload. A consumer that needs order state must track it itself from order_id, or get it from the trading session.

Every match produces a drop copy event per counterparty

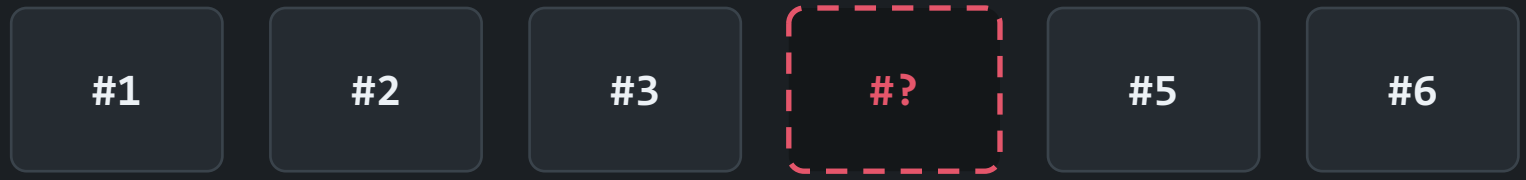
This surprises almost every new operator: the message count is twice the trade count.

The liquidity_flag is derived from which side was the aggressor — it is not something a trader sets.

SEQUENCE NUMBERS

The single field that makes the feed auditable

seq is a process-wide counter: it starts at 1, increases by exactly 1 per event, and never resets while the process lives.



seq 4 never arrived → the consumer knows, immediately and for certain, that it is missing a fill.

1

Detect gaps

Receive 5 straight after 3 and you know 4 was missed — then request a replay or fall back to the audit log.

2

De-duplicate

If a reconnect re-delivers a message, a consumer that remembers processed seq values simply discards the duplicate.

3

Prove completeness

An unbroken run of sequence numbers is evidence for an auditor that nothing was dropped between two points in time.

!

The counter is per process, not per engine instance

Every publisher created inside the same process shares one counter. Build two publishers in one process — as tests and tools sometimes do — and seq keeps climbing rather than restarting at 1.

03

When is it created and disseminated?

Drop copy is not a batch or an end-of-day file. It is produced at the moment of the fill and pushed out immediately.

- The moment of publication
- Which trading flows are covered
- Where it sits in engine startup
- The buffer and the replay window

THE MOMENT OF PUBLICATION

From match to subscriber, in one pass

There is no store-and-forward stage and no scheduler. The publish happens inline as the engine processes the trade.

No batching, no end-of-day

Consumers see fills as they happen, not on a timer.

Fire-and-forget broadcast

The engine does not wait for or track subscriber acknowledgements.

Nothing to poll

A subscriber connects once and messages arrive unprompted from then on.

Because publication is inline, a slow or absent subscriber never slows down matching — it simply misses messages.

COVERAGE

Every fill-producing flow reaches the feed

Drop copy is fed from the engine's single trade-publication path, so it is not limited to simple new-order fills.

- New orders
- Quotes
- Combination legs
- OCO legs
- Auction uncrosses
- Stop cascades
- Amend re-matches
- ...all of them

Why one publication path matters operationally

Because there is exactly one place in the engine where a trade is published, a new trading feature cannot accidentally bypass drop copy. If a fill happened, a drop copy event was emitted for it.

One live event type today

The engine currently emits only order.fill on this feed. The event_type field exists so more types can be added without breaking consumers that already parse the envelope.

ENGINE STARTUP

Drop copy binds at step 8 of 9

Knowing the order tells you what has already happened when the audit feed comes alive.

1. Parse configuration, if a config file is present
2. Bind the main order and broadcast sockets
3. Load persisted book statistics
4. Restore persisted GTC orders
5. Restore persisted GTC combos
6. Inject configured market-maker quotes
7. Inject configured market-maker combos
8. **Bind the drop-copy publisher on :5557, if available**
9. Publish the initial book snapshots

Read it as a checklist

If the engine log shows a drop-copy bind warning at step 8, the exchange will still trade normally — and the audit feed will stay silent all session.

Order has consequences

Restored state and seeded liquidity are already in place before 5557 binds, so those are not announced as fills on the feed. Drop copy describes trading, not initialisation.

THE REPLAY BUFFER

The last 10,000 events are kept in memory

A fixed-size circular buffer: when it is full, the oldest event is discarded to make room for the newest.

10,000

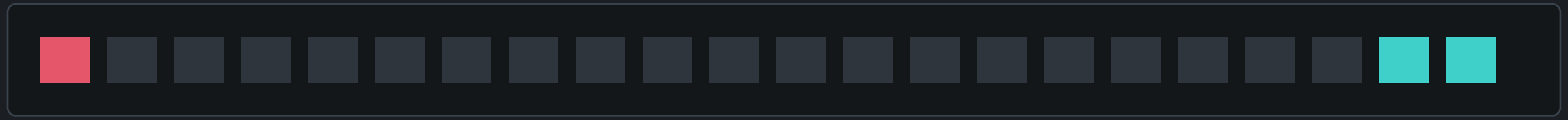
events retained by default
(engine_tuning.drop_copy_buffer_size)

~16 min

of history at a sustained
rate of ~10 fills per second

0

events survive a restart —
the buffer is memory only



oldest — discarded first

newest arrivals

Size the buffer against your worst case, not your average

The window is measured in events, not minutes. A busy session burns through 10,000 events far faster than a quiet one, so a consumer that can be offline for an hour needs either a larger buffer or a durable audit log to fall back on.

REPLAY

Powerful in principle, limited in practice today

Replay re-publishes buffered events on a dedicated topic so a reconnecting consumer can fill its gap.

How a replay is triggered

The publisher exposes a replay call taking a recipient id and a starting sequence number. Every buffered event from that sequence onward is re-published, and the call reports how many messages it sent.

```
drop_copy.replay.<recipient_id>
same payload format as a live event
```

Separate topic by design

Replay traffic uses its own prefix, so one client's catch-up never pollutes the live stream.

Easy to tell apart

Replayed messages are marked as replay and can be consumed alongside live traffic.

There is no over-the-wire replay protocol today

Replay is an in-process call: no external consumer can ask for one, because nothing accepts replay requests. Useful for tests and embedded consumers — not yet a reconnect protocol to rely on.

So what does a real consumer do after a long outage? It reconciles from the durable audit log. Drop copy covers the short gap; persistence covers the long one.

04

What is the format?

The same fill appears in three shapes depending on how you receive it. All three carry the same facts.

- The native wire format
- Topic subscription patterns
- JSON, DC1 text, and the spy line

NATIVE FORMAT

A topic frame and a JSON frame

This is what the engine itself publishes. Everything else in the system is a re-shaping of these two frames.

TWO-FRAME MULTIPART MESSAGE

```
Frame 0  (topic)
drop_copy.event.TRADER01

Frame 1  (payload)
orjson-encoded JSON object
```

- **Text, not binary** The payload is ordinary JSON — readable with any tool, in any language.
- **Flat, not nested** Every field sits at the top level. No sub-objects to walk.
- **Stable across encoders** A faster encoder is used when installed, the standard one otherwise; the bytes are identical.
- **Prices are decimals** fill_price is a display price such as 420.0 — not raw integer ticks.

*** Why the topic is a separate frame**

Subscribers filter on the beginning of the topic frame before the payload is ever examined. Putting the routing key in its own frame is what makes "only TRADER01's fills" cheap to ask for.

Everything downstream — the DC1 text protocol, the spy output, your own subscriber — is a rendering of exactly these fields.

TOPIC PATTERNS

You subscribe to a prefix, not to a query

Filtering is a simple "does the topic start with this text?" test. That single rule gives you three useful patterns.

`drop_copy.event.`

Every live fill, from every gateway

Compliance monitors and observability tools that need the whole picture

`drop_copy.event.TRADER01`

Live fills from one gateway only

A risk system that is responsible for a single participant

`drop_copy.replay.MY_RISK_SYS`

Replay messages addressed to you

Catching up after a reconnect, without disturbing other consumers

Prefer the narrowest prefix that does the job

Subscribing to one gateway rather than the firehose keeps a production consumer's traffic proportional to what it actually needs.

SAME FILL, THREE RENDERINGS

Recognise the same trade wherever you meet it

Native JSON — port 5557

```
{"seq":42,"gateway_id":"TRADER01",
 "event_type":"order.fill",
 "order_id":"ord-001",
 "symbol":"AAPL","fill_qty":100,
 "fill_price":150.05,
 "liquidity_flag":"TAKER"}
```

For programmatic consumers that speak ZeroMQ.

DC1 text — DC Gateway :5590

```
DC_FILL|SEQ=42
 |ORDER_ID=ORD-001
 |SYMBOL=AAPL
 |FILL_QTY=100
 |FILL_PRICE=150.05
 |LIQUIDITY=TAKER
```

For plain TCP clients in any language.

Spy line — pm-dc-spy

```
10:02:17.512  FILL
 TRADER01  AAPL
 #42  100@150.05
 TAKER
 order_id=ord-001
```

For a human reading a terminal.

The mapping is direct

DC1's SEQ and LIQUIDITY come straight from the drop copy envelope and payload. Nothing is added and nothing is invented — the text protocol is a re-encoding, not a richer message.

05

How is ALF related to Drop Copy?

ALF is the order-entry protocol. It can also relay your own drop copy back down the connection you already have.

- Three ways to receive the feed
- The DC toggle on an ALF session
- How real exchanges do the same thing

CHOOSING A PATH

Three ways to receive the same feed

Start from what the recipient already is, and the choice makes itself.

Direct to :5557

`IF: You can speak ZeroMQ`

- Most general option
- See all gateways or filter by prefix
- Right for an independent risk, clearing or compliance system

DC relay on ALF

`IF: You already have a trading session`

- No second connection to manage
- Scoped to your own gateway id only
- Closest match to how real venues deliver drop copy

DC Gateway :5590

`IF: Plain TCP, no ZeroMQ available`

- Any language with a sockets library
- Simple text lines, one per fill
- Right for external partners and other hosts

They are not mutually exclusive

All three read the same feed and can run at once. Turning on the DC relay for your ALF session does not stop a separate subscriber or spy from also watching port 5557.

THE ALF RELAY

Turning your own fills on inside an existing session

One command, and every fill for that gateway starts arriving asynchronously on the connection you already have open.

ENABLING THE RELAY

```
# enable from the start
pm-alf-console --id TRADER01 --drop-copy

# or toggle at any time, on console or a TCP client
DC|STATE=ON
```

WHAT THEN ARRIVES, UNPROMPTED

```
DC_FILL|SEQ=42|ORDER_ID=ORD-001|SYMBOL=AAPL
       |FILL_QTY=100|FILL_PRICE=150.05|LIQUIDITY=TAKER
```

It is the same feed underneath

The gateway or console subscribes to your gateway's drop copy topic on your behalf and re-delivers it down your existing connection. Nothing new is generated.

Scoped to you, by construction

The relay only asks for your own gateway id, so a session sees its own fills. Note this is scoping, not enforcement — see the security slide in section 8.

*

When to prefer the relay

A participant who simply wants a durable record of their own executions gets it with one command and no extra infrastructure.

REALITY CHECK

How production exchanges publish drop copy

Worth knowing, because the vocabulary you will meet in the industry comes from this model.

	A real venue	EduMatcher
Delivery	A dedicated FIX session per recipient, separate from the trading session	EduMatcher: a ZeroMQ broadcast, or a relay on your existing session
Payload	An Execution Report — the same message type used for live acks and fills	EduMatcher: a small JSON object, or a DC1 text line
Sequencing	Session-scoped FIX sequence numbers	EduMatcher: a process-wide counter carried in seq
Gap recovery	FIX's native resend request and sequence reset	EduMatcher: an in-process replay from a memory buffer
Entitlement	Structural — the session itself is provisioned per participant	EduMatcher: none on the feed; scoping is by topic only

*

Which EduMatcher path is the better simulation?

The ALF relay — a per-participant feed layered on an existing session. The raw port-5557 connection is closer to tapping the wire between the exchange and its drop copy relay: ideal for building tooling, not how a participant would normally be served.

06

How does a client use the DC Gateway?

A gateway is a door: it translates an internal feed into a protocol an outside system can actually speak.

- What a gateway is for
- The pm-dc-gwy topology
- The session handshake
- The DC1 message set

WHAT A GATEWAY IS FOR

Three jobs, in every gateway you will meet

Before the specifics of pm-dc-gwy, the general idea — because the same pattern repeats across ALF, BALF, CALF and RALF.

1

Translate the protocol

Inside, the exchange speaks its own message bus. Outside, clients speak plain TCP text. A gateway converts between the two so neither side has to change.

2

Manage the session

Connections need a handshake, heartbeats, an idle policy and a clean disconnect. The gateway owns all of that so the feed itself stays simple.

3

Protect the core

It absorbs slow clients, bad input and disconnects at the edge, keeping that behaviour away from the matching engine.

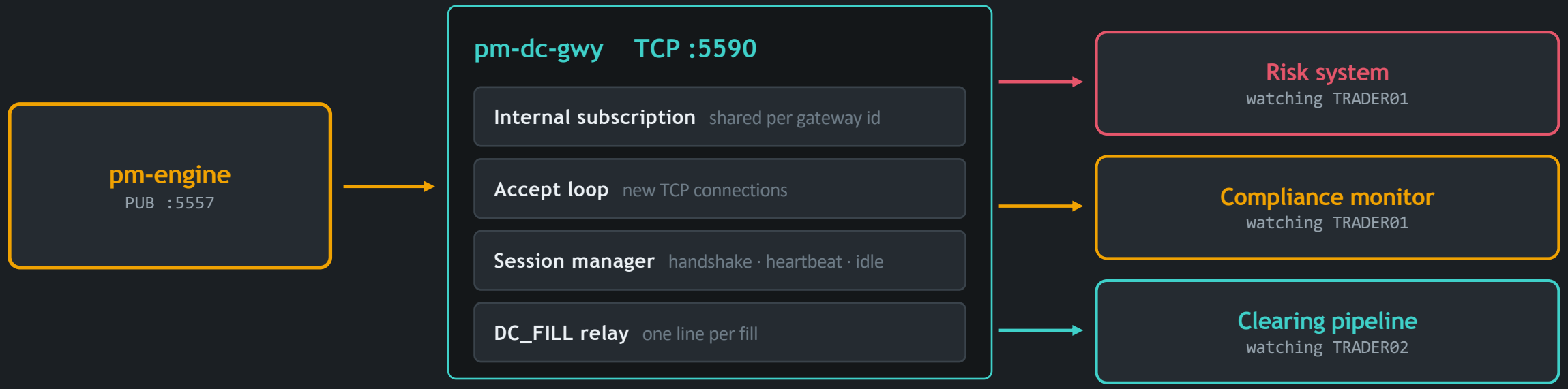
Applied to Drop Copy

pm-dc-gwy subscribes to the engine's drop copy feed on your behalf and relays it as DC1 text lines to any number of connected TCP clients — each scoped to whichever gateway id it asked for at connect time. It is deliberately the simplest protocol in the system: there is exactly one thing a client can ask for.

TOPOLOGY

One subscription in, many clients out

The gateway holds a single internal subscription per gateway id and fans it out — clients watching the same participant share it.



Two things this buys you

More than one client may watch the same gateway id at once — unlike a trading session, a drop copy connection does not occupy a trading identity. And the engine still sees only one subscriber, however many clients connect.

WHAT IT DELIBERATELY DOES NOT DO

A read-only relay, and nothing more

Not supported	Why
Authentication or entitlement checks	Same scope decision as the raw feed — any client may request any gateway id
Replay by sequence over the wire	Replay is in-process only with no external trigger, so the gateway does not expose it
Order entry, quoting, or any state change	This is a read-only relay — use the trading gateways for anything that mutates state
A role or channel model	Roles such as CLEARING and AUDIT belong to RALF's separate mechanism

So what can a client ask for?

Exactly one thing: fills for a given gateway id. That is why DC1 has no subscription grammar to learn.

Prerequisite before you start it

pm-engine must be running with its drop copy publisher bound on 5557. The gateway relays a feed — it does not create one.

SESSION LIFECYCLE

Four steps from connect to fills

Validate the whole flow with a terminal before writing a single line of client code.

1	**Connect**	Open a TCP connection to the gateway port	`nc 127.0.0.1 5590`
2	**Say HELLO**	The very first line must be a HELLO, naming a client label, the protocol, and the gateway id you want	`HELLO|CLIENT=risksys|PROTO=DC1|ID=TRADER01`
3	**Get WELCOME**	The gateway subscribes you, then confirms with your id, its name, the heartbeat interval and idle timeout	`WELCOME|PROTO=DC1|GW=dc-gwy01|ID=TRADER01|HBINT=5|IDLE=30`
4	**Receive fills**	No further command is needed — every fill for that id arrives unprompted for as long as you stay connected	`DC_FILL|SEQ=42|ORDER_ID=ORD-001|SYMBOL=AAPL|FILL_QTY=100|...`

Send EXIT to close cleanly. The gateway then drops its internal subscription if no other client still wants that gateway id.

THE DC1 MESSAGE SET

Three things you can send, five you can receive

All lines share one shape: a message type, then pipe-separated FIELD=VALUE pairs, terminated by a newline.

CLIENT → GATEWAY

Message	Fields	Notes
HELLO	CLIENT, PROTO, ID	Must be the first line
PING	—	Gateway replies PONG
EXIT	—	Graceful disconnect

GATEWAY → CLIENT

Message	Fields	Trigger
WELCOME	PROTO, GW, ID, HBINT, IDLE	Reply to a valid HELLO
DC_FILL	SEQ, ORDER_ID, SYMBOL, FILL_QTY, FILL_PRICE, LIQUIDITY	One per fill, unsolicited
PONG	TS	Reply to PING
HB	TS	Heartbeat when otherwise silent

The mistake every first-time client makes

TCP is a byte stream, not a stream of lines. One read is not one message: it may deliver half a line, or three at once. Always append to a buffer and split on newline. Also send a PING periodically, or the gateway will close your idle connection.

07

What is the DC Spy client for?

pm-dc-spy answers one question fast: what is the engine actually publishing right now?

- What it is, and what it is not
- The flags that matter
- Reading its output
- Everyday recipes

A read-only window onto the feed

It connects to the drop copy port, subscribes for one gateway or all of them, and prints everything it receives.

?

What it answers

"Is anything being published at all, and does it contain what I expect?" — without writing a subscriber first.

=

Safe by construction

It never publishes anything back. Run as many instances as you like: the feed fans out independently to every subscriber.

#

Machine-readable too

A JSON mode makes it a capture tool: pipe it into a filter, or redirect it to a file for later analysis.

Not a protocol client — this trips people up

No handshake, no heartbeat

There is no session to establish, so there are no client-name, ping-interval or show-heartbeat flags.

No WELCOME banner

The connection line you see is printed locally by the tool — the engine did not send it.

It never fails to "connect"

Subscribing succeeds even if the engine is not running yet; it starts delivering once a publisher appears.

Consequence: silence on screen means "no fills yet", not "not connected". Cause a trade to prove the path.

READING THE OUTPUT

One line per fill, in a fixed order

Learn the column order once and you can scan a session at a glance.

HUMAN FORMAT — THE TWO SIDES OF ONE TRADE

```
10:02:17.512  FILL    TRADER01   AAPL      #1     100@150.05      TAKER  order_id=ord-001
10:02:17.520  FILL    TRADER02   AAPL      #2     100@150.05      MAKER  order_id=ord-104
```

Time	Kind	Gateway	Symbol	#Seq	Qty@Price	Liquidity
local wall clock	FILL or REPLAY	whose fill it is	instrument	sequence number	the execution	MAKER or TAKER

Colour tells you the side

MAKER and TAKER are colour-coded, so the shape of a busy session is visible without reading every word.

Add the raw payload

A raw option echoes the exact topic and JSON under each line — ideal when you are verifying a field by eye.

Replays stand out

Replayed messages read REPLAY instead of FILL, so catch-up traffic is never mistaken for live trading.

EVERYDAY RECIPES

Four commands worth memorising

Watch one participant live	`pm-dc-spy --gateway TRADER01`	The everyday check when a trader reports a missing fill.
Capture a sample for analysis	`pm-dc-spy --format json --count 100 > fills.jsonl`	Stops itself after 100 messages — safe to run in a script.
Filter by symbol	`pm-dc-spy --format json | jq 'select(.symbol == "AAPL")'`	JSON mode preserves every field, so any filter you like works.
Watch a replay happen	`pm-dc-spy --replay-of MY_RISK_SYS`	You cannot trigger a replay from here — but you can see exactly what one re-publishes.

Independent instances do not interfere — run one per gateway across several terminals instead of filtering one stream by eye.

08

Good to know when using Drop Copy

The facts that separate an operator who trusts the feed from one who is surprised by it.

- Eight operating truths
- The security reality
- A pre-flight checklist

OPERATING TRUTHS

Eight facts to internalise before you rely on the feed

01

Two events per trade

Count messages, not trades — every match emits one event per counterparty.

02

Order state is not included

No side, no remaining quantity. Track it yourself or read it from the trading session.

03

The counter is per process

Sequence numbers are shared process-wide and never reset while the process lives.

04

Silence is ambiguous

No messages can mean no trading, or a publisher that never bound. Check the engine log.

05

The buffer is memory only

A restart empties it. There is no persistence behind drop copy itself.

06

Replay is not yet a protocol

No external consumer can request one over the wire today.

07

Slow clients get dropped

Fall too far behind on the DC Gateway and your connection is closed, not throttled.

08

Prices are display decimals

fill_price is a decimal price, not raw integer ticks — do not rescale it.

SECURITY

There is no authentication on this feed

This is a deliberate scope decision in an educational system — and it is the fact you must never carry into a production mindset.

What is not checked

- Any process that can reach the drop copy port may subscribe to every gateway's fills
- The gateway id in a DC1 HELLO is not validated against any allowlist
- The gateway filter in the spy tool is a readability convenience, not access control

How a real venue closes the gap

- Entitlement is structural: the drop copy session itself is provisioned per participant
- You cannot ask for someone else's fills, because you have no session that carries them
- The RALF channels in this system do enforce a per-connection role, and are the closer analogue

! **What this means for how you deploy it**

Treat the drop copy port and the DC Gateway as trusted-network services. Bind them to a local interface unless you have a reason not to, and put network controls — or a proxy that terminates TLS — in front of anything that must reach another host.

The gateway does not terminate TLS itself. For remote deployments, place it behind a reverse proxy.

09

What errors can the feed give?

Errors arrive as a single line carrying a code and a detail. Four codes cover everything the DC Gateway will tell you.

- The four error codes
- Symptom to cause to fix
- The silent-feed decision path

ERROR CODES

Four codes, and what each one costs you

Every error arrives in the same shape: an error line with a CODE and a human-readable DETAIL.

```
ERR|CODE=<CODE>|DETAIL=<message>
```

Code	When it happens	Connection kept?	What to do
AUTH_REQUIRED	First line was not a HELLO, or the HELLO was missing a required field	No — closed at once	Send a complete HELLO as the very first line
BAD_MESSAGE	An empty line, invalid text encoding, an unparseable line, or a line over 4096 bytes	Usually yes	Check your line framing — never send a bare newline
SLOW_CLIENT	Your outbound queue exceeded the configured per-client limit	No	Consume faster, or raise the queue limit in config
IDLE_TIMEOUT	No inbound traffic from you for the configured idle period	No	Send a PING periodically, or raise the idle timeout

Note what is missing: there is no "unknown gateway" and no "not entitled" error — because neither is checked.

TROUBLESHOOTING

Symptom, likely cause, fix

The five problems you will actually meet, in the order you are likely to meet them.

Symptom	Likely cause	Fix
Connection refused	The gateway is not started, or you are on the wrong port	Confirm the process is running; check the configured port
An auth error immediately on connect	The first line was not a HELLO, or a required field was missing	Send the full HELLO with a client label, the protocol and an id
WELCOME arrives but no fills ever do	No trades have occurred for that gateway id — or the engine's publisher failed to bind	Check the engine log for a bind warning; confirm trades are happening for that id
The gateway hangs up after about 30 seconds	The idle timeout elapsed with no inbound traffic	Send a PING on a timer, or raise the idle timeout in configuration
Unreachable from another host	The gateway is bound to the loopback interface only	Bind to all interfaces, or to the specific interface address

Two commands settle most of these: check who is listening on the port, then drive a HELLO through a raw TCP tool before blaming the client code.

A decision path for "no messages are arriving"

Work down it in order — each step rules out one whole class of cause.

1	**Is the engine running and did it log a drop copy bind warning at startup?**	A failed bind leaves the exchange trading with no audit feed at all.
2	**Is anything actually trading right now?**	The feed only speaks when a fill occurs. Cause a trade to prove the path end to end.
3	**Are you subscribed to the right prefix?**	Filtering for one gateway id that never trades looks identical to a broken feed.
4	**Can you see fills with the spy tool?**	If the spy sees them and your consumer does not, the problem is in your client, not the feed.
5	**If using the DC Gateway: did you get a WELCOME?**	No WELCOME means the handshake failed. A WELCOME with no fills points back to steps 2 and 3.

Remember: with the spy tool, connecting always succeeds — so an empty screen never proves you are connected to the right place.

10

What configuration affects Drop Copy?

Three places hold every knob that changes how drop copy is produced, buffered and relayed — plus one that is not in the config file at all.

- How configuration reaches a running exchange
- The gateway settings, field by field
- The engine tuning knob that sizes the buffer
- The address constant that is not in YAML

What you author is not what the exchange runs

Understanding this pipeline prevents the classic failure: editing a file that nothing reads.

Author it under version control

The YAML lives wherever suits you — edit it, review it, diff it like any other source file.

Deploy compiles and checks it

Validation runs, every default is resolved exactly once, and the result is installed atomically.

One file, read by everything

No process takes a config path, so it is impossible to start two of them against different configurations.

The mistake this design prevents — and the one it does not

Processes can no longer disagree about configuration. But editing the YAML and forgetting to deploy still changes nothing at all.

WHERE THE DROP COPY SETTINGS LIVE

Three separate homes, on purpose

dc_gateway

Top-level section, optional

Read by the DC Gateway process only. Bind address, port, heartbeat, idle timeout and the per-client queue limit.

Affects how clients connect to the relay.

engine_tuning

Top-level section, optional

Read by the engine. Contains drop_copy_buffer_size, which sets how many events are kept for replay.

Affects how much history the engine remembers.

The publish address

Not in the config file

The drop copy publish address is a constant in the codebase, overridable per run on the relay's command line.

Affects where the feed is published at all.

*

One more thing worth knowing

The engine's own gateway allowlist controls which trading ids may connect and submit orders. It does not gate who may read drop copy — those are separate concerns.

Every field, and why you would change it

All fields are optional and fall back to the defaults shown. Generate the block with the config generator, or hand-edit it.

DEFAULTS, IN FULL

```yaml
dc_gateway:
  name: "dc-gwy01"
  bind_address: "0.0.0.0"
  port: 5590
  heartbeat_interval_sec: 5
  idle_timeout_sec: 30
  max_client_queue: 10000
```

Field	Default	Why you would change it
name	dc-gwy01	Identifies the process in the WELCOME line — useful when you run more than one
bind_address	0.0.0.0	Set to the loopback address to keep the relay local-only; the default accepts remote clients
port	5590	Change on a port clash, or to run a second relay alongside the first
heartbeat_interval_sec	5	How often the gateway proves it is alive when there are no fills to send
idle_timeout_sec	30	Raise it for clients that cannot ping regularly — or they will be disconnected
max_client_queue	10000	How far a client may fall behind before it is dropped as slow

The two that bite

bind_address decides whether anyone off-host can reach you. idle_timeout_sec decides whether a quiet client survives.

THE BUFFER KNOB

engine_tuning.drop_copy_buffer_size

One integer that decides how much history the engine can replay — and how much memory that costs.

```
engine_tuning:
  drop_copy_buffer_size: 10000
  # must be an integer greater than zero
  # defaults to 10000 when omitted
```

Also settable on the generator

The config generator exposes it as a drop-copy buffer size option, so it can be produced rather than hand-written.

Remember the unit: the buffer holds a number of events, not a span of time. The same setting covers very different windows in a quiet and a busy session.

Too small

A consumer that reconnects after a brief outage finds its starting sequence has already been discarded, and silently receives only part of what it missed.

Too large

Memory grows for history nobody asks for — and replay is in-process only today, so most of that history is never read.

Sizing it honestly

Estimate your peak fills per second, multiply by the longest outage you want to survive, and set the buffer above that number.

Why the publish address behaves differently

Every other setting here lives in the configuration file. This one does not — and the reason matters.

```
# engine_config.yaml — no drop copy section is needed
# the address is a constant in the codebase:
DROP_COPY_PUB_ADDR = "tcp://127.0.0.1:5557"
```

One constant, used by everyone

The engine, the relay gateway and the spy tool all resolve the drop copy address from the same place. There is no way for them to drift apart into disagreeing about where the feed lives.

To change it permanently

Edit that constant in the codebase — a deliberate act, not a per-deployment tweak.

Per-run overrides on the tools

`--engine-dc-pub ADDR`
On the DC Gateway: subscribe to a non-default engine address

`--host / --port`
On the spy tool: point at another host or port

`--bind / --port`
On the DC Gateway: override the TCP listen address for one run

`-v / -vv / --log-level`
Raise verbosity when you are diagnosing a silent feed

Overrides last one run. Configuration lasts until you change it.

A working configuration, end to end

The minimum a new operator needs to get a functioning, observable drop copy path.

FROM EMPTY CONFIG TO A VERIFIED FEED

```
# 1 — generate or hand-edit the relay section
pm-config-gen --dc-gateway --dc-port 5590 \
              --dc-idle-timeout-sec 60

# 2 — validate, compile and install it
pm-config-deploy my_config.yaml

# 3 — start the engine, then the relay
pm-engine --verbose
pm-dc-gwy

# 4 — prove the feed is alive
pm-dc-spy --gateway TRADER01
```

Verify in this order

- Engine log shows no bind warning
- The relay is listening on its port
- A HELLO returns a WELCOME
- A test trade produces two events
- Sequence numbers advance by one

Do step 4 every time

A configuration that parses is not a feed that flows. Only a real fill proves the path.

IN SUMMARY

What to carry away

It is the exchange's second telling

A copy of every fill, on a channel built for risk, audit and clearing rather than for trading.

Its value is completeness

Sequence numbers, not speed, are what make it trustworthy — gaps become visible instead of invisible.

Three ways in, one feed

Direct subscription, an ALF session relay, or the plain-TCP gateway. Pick by what the recipient already speaks.

Configuration decides whether it exists

A failed bind, a loopback-only relay or an undersized buffer all leave trading healthy and the audit trail incomplete.

If trading works but nobody downstream can prove what happened, the system is not correctly configured.